Student Lab: First Day at CyberX Coffee

Oct 21, 2026 · @Reza

Welcome to CyberX

Today is your first day as the junior sysadmin at **CyberX Coffee Roasters**, a small Munich roastery with an online shop called BrewBot. The previous admin, Sam, left in a hurry. He handed over a messy folder, an unfinished to-do list and a few surprises. Your manager, Marta from ops, needs it all sorted by 5 pm.

How this lab works

- Your instructor works through each mission on the projector. Type along on your own machine.
- Lines in grey boxes are commands — type them yourself rather than copy-pasting. You learn more that way.
- **Checkpoint** lines tell you what you should see. If yours looks different, raise your hand.
- **Your answer** lines are for you to fill in. Edit this doc or write in a notebook.
- Work with your neighbour. Two pairs of eyes catch typos fast.

Getting started

You'll do the whole lab in your own Ubuntu 26.04 virtual machine. Before the session, follow **Get the lab onto your VM** below. At the start of class, open a terminal in your VM (`Ctrl+Alt+T`) and check that your lab is there:

```
ls ~/cyberx
```

Checkpoint: `ls ~/cyberx` lists `HANDOVER.txt`, `bin`, `data` and more.

If anything goes badly wrong at any point, rebuild your lab with `cyberx-setup` (your `work` folder will be lost).

When you're stuck

- Press `Ctrl+C` to stop a running command.
- Press `q` to leave `less`, `man` or `top`.
- In `vim`, press `Esc` twice, then type `:q!` and Enter to escape without saving.

Get the lab onto your VM (before class, 5 min)

You need one file, `cyberx-setup.sh`, inside your Ubuntu 26.04 VM. When you run it, it builds a folder called `cyberx` in your home directory with everything the missions use. Do every step **inside the VM**, logged in as the user you created when you installed Ubuntu.

Step 1 — Start your VM and log in. Check that the VM has internet: open Firefox in the VM and load any website.

Step 2 — Download the setup script inside the VM. Pick one way:

- **With the browser (easiest):** in the VM, open Firefox, go to the link your instructor shared (course page, LMS or email) and download `cyberx-setup.sh`. Firefox saves it in your `Downloads` folder.
- **With the terminal:** if your instructor gave you a direct download link, open a terminal (`Ctrl+Alt+T`) and run the two commands below, with the link in place of `<link>`.

```
cd ~/Downloads
wget -O cyberx-setup.sh "<link>"
```

Download the file inside the VM, not on your own computer. That way you don't need shared folders or drag-and-drop between your computer and the VM.

Step 3 — Check you have the right file.

```
cd ~/Downloads
ls -l cyberx-setup.sh
head -n 3 cyberx-setup.sh
```

Checkpoint: the file is about 12 KB, and the third line mentions "First Day at CyberX Coffee". If Firefox saved it under another name, such as `cyberx-setup(1).sh`, use that name in the next step.

Step 4 — Run it. Type `bash` and the file name. Don't put `sudo` in front.

```
bash cyberx-setup.sh
```

The first time, it asks for your password (the one you log in with). It uses it once to install a few tools the lab needs, such as `vim`, and to install the command `cyberx-setup`. Ubuntu shows a `*` for each character you type.

Checkpoint: the last lines say `CyberX lab ready in /home/<you>/cyberx`.

Step 5 — Confirm everything is ready.

```
ls ~/cyberx
cyberx-setup --check
```

Checkpoint: `ls` lists `HANDOVER.txt`, `bin`, `data`, `disk`, `etc` and `logs`, and `--check` prints `All tools present.`

If something goes wrong:

You see	Fix
`bash: cyberx-setup.sh: No such file or directory`	You're in a different folder. Run `cd ~/Downloads`, then `ls` to find the file.
"Please run this as your normal user"	You typed `sudo` in front. Run it again without `sudo`.
"could not install … (no internet?)"	Your VM has no internet. Fix the VM's network, then run `bash cyberx-setup.sh` again.
"Missing packages: …" from `--check`	Run the `sudo apt install -y …` command it prints.
"Manual pages are missing"	Run `sudo unminimize`, answer `y`, then try again.
`cyberx-setup: command not found`	The install step was skipped. Wherever this guide says `cyberx-setup`, type `bash ~/Downloads/cyberx-setup.sh` instead.

Mission 0 – Badge in (10 min)

8:30. Marta hands you a laptop. "Before you touch anything, tell me what you're actually logged into."

0.1 — Which Linux is this?

```
cat /etc/os-release
uname -r
uname -a
```

Your answer: distribution and version: _____ · kernel version: _____

0.2 — Which shell am I in?

```
echo $SHELL        # your login shell
echo $0            # the shell running right now
cat /etc/shells    # shells installed on this system
```

Your answer: my shell is _____. Other shells installed: _____

0.3 — My environment. Variables hold settings that programs can read.

```
echo $HOME $USER
echo $PATH
env | less
CYBERX=~/cyberx
bash -c 'echo "child sees: $CYBERX"'
export CYBERX
bash -c 'echo "child sees: $CYBERX"'
```

Your answer: what changed between the two `bash -c` lines, and why? _____

0.4 — Getting help. Later today you'll need to compress files, but you don't know the command yet.

```
man -k compress
man gzip           # q quits, /word searches, n finds next
gzip --help
type cd ls
```

Your answer: a command that compresses files: _____. Is `cd` a builtin or a program? _____

> Habit 1: **man first.** Look it up before you guess.

Mission 1 – Find Sam's files (15 min)

Marta: "Sam left everything in ~/cyberx. Read his note, find out what's in there, and set up your own work area. Don't break anything of his."

1.1 — Get there and read the note.

```
cd ~/cyberx
pwd
ls
cat HANDOVER.txt
less HANDOVER.txt     # space = next page, b = back, q = quit
```

Checkpoint: `pwd` prints a path ending in `/cyberx`. The note lists 5 unfinished tasks — those are your missions today.

1.2 — Find the secret notes. Sam says there's a private file "if you look properly".

```
ls -a
ls -alF
cat .sam_notes
```

Your answer: why didn't plain `ls` show it? _____. What do `/` and `*` mean after a name? _____

1.3 — Map the whole tree.

```
ls -alFR | less
```

1.4 — Identify the mystery files. They have no extensions.

```
file data/*
```

Your answer: mystery1 is _____ · mystery2 is _____ · mystery3 is _____

1.5 — Build your work area and back up the config before you change it.

```
mkdir -p ~/cyberx/work/reports ~/cyberx/work/backup
touch work/reports/day1.md
cp -i etc/brewbot.conf work/backup/brewbot.conf.orig
ls -l work/*
```

1.6 — Rename and tidy.

```
mv -i work/reports/day1.md work/reports/day1-notes.md
rmdir temp_do_not_delete
rmdir work           # try it — what happens?
```

Your answer: why did the last `rmdir` refuse? _____

1.7 — Links. A shortcut to the log you'll read all day, and a hard link to compare.

```
ln -s ~/cyberx/logs/brewbot.log ~/current.log
ln data/customers.txt work/customers.hard
ls -li ~/current.log data/customers.txt work/customers.hard
```

Your answer: which two files share an inode number? _____. If the original were deleted, which link would still work? _____

> Habit 2: **pwd after cd.** Habit 3: **-i before you overwrite or delete.**

Mission 2 – Fix the config in vim (20 min)

Marta: "BrewBot's config is wrong. Port must be 8080, log level warn, alerts go to ops@cyberx.example. And get rid of that TEMP_HACK line."

vim has three modes. Most confusion comes from not knowing which one you're in.

Mode	You're in it when…	Get in	Get out
Normal	you open a file; keys are commands	`Esc`	`i` , `a` , `o` → Insert; `:` → Ex
Insert	you're typing text	`i` insert, `a` append, `o` new line	`Esc` → Normal
Ex	the cursor sits on the bottom line after `:`	`:` from Normal	`Enter` runs it, `Esc` cancels

Lost? Press `Esc` twice — you're back in Normal mode.

2.1 — Look around without changing anything.

```
vim etc/brewbot.conf
```

Move with `h j k l` or the arrow keys. `G` jumps to the end, `gg` to the top. Leave with `:q`.

2.2 — Make the four fixes. Your toolkit:

Keys	Does
`/80800` + `Enter`	search for text

Keys	Does
`cw`	change word (deletes to word end, enters Insert)
`x` / `dd`	delete a character / a whole line
`A`	append at end of line
`u` / `Ctrl+r`	undo / redo

- [ ] `listen_port=8080`
- [ ] `log_level=warn`
- [ ] `alert_email=ops@cyberx.example`
- [ ] `TEMP_HACK` line deleted

2.3 — Save. `:w` saves and keeps the file open. `:wq` (or `ZZ`) saves and quits.

2.4 — The emergency exit. Open the file again, delete three lines with `3dd`, then quit **without saving** with `:q!`. Prove nothing was lost:

```
diff work/backup/brewbot.conf.orig etc/brewbot.conf
```

Checkpoint: `diff` shows only your four fixes from 2.2.

Bonus: in Ex mode try `:%s/CyberX/CYBERX/g`, look, then `u` to undo. **Your answer:** what do `%` and `g` mean? _____

Mission 3 – Marta's report (10 min)

Marta: "I need to know which products sold best last week. There's no Excel on the server, so… figure it out."

A **filter** reads text in, changes it, and writes text out. Chain filters with `|` and you can answer almost any question about a text file.

Viewing

3.1 — First look.

```
head -n 5 data/orders.csv
tail -n 3 data/orders.csv
```

Your answer: which dates does the export cover? _____ to _____

Summarizing

3.2 — How many orders?

```
wc -l data/orders.csv
```

Your answer: number of orders (careful — line 1 is a header): _____

3.3 — Fingerprint it so Marta can check your copy matches hers.

```
md5sum data/orders.csv
```

Checkpoint: compare with your neighbour. The checksums should be identical.

3.4 — Top sellers (main deliverable). Build this one step at a time. Run each line and watch what each new filter adds.

```
tail -n +2 data/orders.csv | cut -d, -f3 | head
tail -n +2 data/orders.csv | cut -d, -f3 | sort | head
tail -n +2 data/orders.csv | cut -d, -f3 | sort | uniq -c
tail -n +2 data/orders.csv | cut -d, -f3 | sort | uniq -c | sort -rn
```

Your answer: #1 product _____ (____ orders) · #2 _____ · #3 _____

Bonus: which customer ID placed the most orders? (Hint: same pipeline as 3.4, but field 2.) _____

Mission 4 – The declines (20 min)

The payment provider emails: "Your shop sent us an unusual number of declined payments yesterday." Marta: "Is it us, or is someone attacking the checkout?"

`grep` prints the lines that match a pattern. A **regular expression** is a pattern language:

Pattern	Means	Basic (`grep`)	Extended (`grep -E`)
`^` / `$`	start / end of line	same	same
`.`	any one character	same	same
`[0-9]`	one character from a set	same	same
`*`	previous item, 0 or more times	same	same
`+` / `?`	1 or more / 0 or 1	`\+` / `\?`	`+` / `?`

Pattern	Means	Basic (`grep`)	Extended (`grep -E`)
`{n,m}`	n to m times	`\{n,m\}`	`{n,m}`
`|`	either / or	`|`	`|`
`( )`	group	`\( \)`	`( )`

You can use your shortcut `~/current.log` from Mission 1 in place of `logs/brewbot.log`.

Using grep

4.1 — How bad is it?

```
grep -c ERROR logs/brewbot.log
grep -ci error logs/brewbot.log
grep -i error logs/brewbot.log | grep -v ERROR | head -3
```

Your answer: ERROR count ____ · case-insensitive count ____ · why the difference?

_____

4.2 — Just the declines, with line numbers.

```
grep -n declined logs/brewbot.log | less
```

4.3 — Everything except the noise.

```
grep -v 'health check' logs/brewbot.log | grep -v INFO | less
```

Basic regular expressions

4.4 — The morning rush (7:00–8:59).

```
grep -c '^2026-10-20 0[78]:' logs/brewbot.log
```

Your answer: lines during the rush: _____

4.5 — Lines ending in `30s`.

```
grep '30s$' logs/brewbot.log | head
```

Extended regular expressions

4.6 — Problems at a glance.

```
grep -cE 'WARN|ERROR' logs/brewbot.log
```

4.7 — Find the attacker (main deliverable). Build it step by step again:

```
grep declined logs/brewbot.log | head -3
grep declined logs/brewbot.log | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' |
head -3
grep declined logs/brewbot.log | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' |
sort | uniq -c | sort -rn | head -3
```

Your answer: the attacking IP is _____ with ____ declines.

In plain words, what does `([0-9]{1,3}\.){3}[0-9]{1,3}` describe? _____

4.8 — Really slow responses (1500 ms or more).

```
grep -E 'slow response 1[5-9][0-9]{2} ms' logs/brewbot.log | head
```

Bonus: list every unique customer email address in the log. How many are there? _____
(You'll hide these in Mission 6.)

Mission 5 – The runaway process (15 min)

11:40. Marta rushes over: "The shop is crawling and a file called runaway.log just appeared. Sam's notes mention brewlogd. Find it and stop it — all of it."

Start the incident (only when your instructor says so):

```
cyberx-setup --incident
```

A **process** is a running program. Each one has a PID (its ID) and a PPID (its parent's ID). To stop one, you send it a signal: `15` (TERM) asks politely and is the default; `9` (KILL) forces it.

5.1 — Watch it happen.

```
tail -n 3 logs/runaway.log
tail -f logs/runaway.log      # Ctrl+C stops watching (not the problem!)
```

5.2 — Find the culprit.

```
ps -ef | grep brewlogd
ps -ef --forest | less        # type /brewlogd to search, q to quit
```

Your answer: how many copies are running? ____ · their PIDs: _____ · their parent PID: ____

5.3 — Live view.

```
top
```

Press `P` to sort by CPU, `M` by memory, `q` to quit.

5.4 — Stop one politely. Replace `<PID>` with one of your PIDs from 5.2.

```
kill <PID>
ps -ef | grep brewlogd
```

Checkpoint: one copy is gone, the others are still running.

5.5 — Stop them all by name.

```
killall brewlogd
ps -ef | grep brewlogd
ls -l logs/runaway.log
```

Wait 5 seconds and run `ls -l logs/runaway.log` again.

Checkpoint: only the `grep` line itself is left in the `ps` output, and the file size no longer changes.

Your answer: why should `kill -9` be your last resort, not your first? _____

Mission 6 – The disk is full (25 min)

Marta: "Free up space, archive the old logs, and send the vendor yesterday's log — without customer emails, or legal will have my head."

Every program has three **streams**: standard input (0), standard output (1) and standard error (2). You can point them wherever you like:

Syntax	Does
`cmd > file`	output to file (overwrites)

Syntax	Does
`cmd >> file`	output to file (appends)
`cmd 2> file`	errors to file
`cmd &> file`	output and errors to file
`cmd < file`	file becomes the input
`cmd1 | cmd2`	output of cmd1 becomes input of cmd2
`cmd | tee file`	show on screen *and* save to file

Where did the space go?

6.1 — Disk overview.

```
df -h
df -h ~
```

Your answer: free space where your home directory lives: _____

6.2 — What's big?

```
du -sh ~/cyberx/*
du -sh ~/cyberx/* | sort -h
```

Your answer: the biggest directory is _____ at _____

Redirecting input and output

6.3 — Start a disk report.

```
du -sh ~/cyberx/* > work/reports/disk.txt
df -h ~ >> work/reports/disk.txt
cat work/reports/disk.txt
```

Your answer: what would have happened with `>` instead of `>>` on the second line?

_____

6.4 — Separate the errors.

```
du -sh /etc/*
du -sh /etc/* > work/reports/etc-sizes.txt 2> work/reports/errors.txt
du -sh /etc/* 2> /dev/null
```

Piping data between programs

6.5 — Save the attacker report and see it too.

```
grep declined logs/brewbot.log | grep -Eo '([0-9]{1,3}\.){3}[0-9]{1,3}' |
sort | uniq -c | sort -rn | head -3 | tee work/reports/attackers.txt
```

Using sed

6.6 — Redact for the vendor (main deliverable).

```
sed -E 's/[a-z.]+@[a-z.]+/[REDACTED]/g' logs/brewbot.log | head -3
sed -E 's/[a-z.]+@[a-z.]+/[REDACTED]/g' logs/brewbot.log >
work/reports/vendor.log
grep -c '@' work/reports/vendor.log
```

Checkpoint: the last command prints `0`.

6.7 — More sed.

```
sed -n '10,15p' logs/brewbot.log
wc -l work/reports/vendor.log
sed -i '/DEBUG/d' work/reports/vendor.log
wc -l work/reports/vendor.log
```

Your answer: how many DEBUG lines were removed? _____

Generating command lines and archiving

6.8 — Archive the old logs. `$( )` runs a command and drops its output into your command line.

```
echo "Today is $(date +%F)"
tar -czvf work/backup/old-logs-$(date +%F).tgz logs/old
tar -tzvf work/backup/old-logs-*.tgz
ls -lh work/backup
du -sh logs/old
```

Your answer: original size _____ · archive size _____

6.9 — Compress with xargs. `xargs` turns a list of names into arguments for another command.

```
ls logs/old/brewbot.log.*
ls logs/old/brewbot.log.* | xargs gzip -v
zcat logs/old/brewbot.log.1.gz | head -3
```

6.10 — Clean up safely. The archive is verified, so the originals can go. Answer `y` to each prompt.

```
du -sh ~/cyberx
rm -ri logs/old
rm -i logs/runaway.log
du -sh ~/cyberx
```

Your answer: space freed: _____

6.11 — The 2023 receipts. Mount Sam's old disk image, read the December total, then unmount it. `sudo` asks for your password.

```
sudo mkdir -p /mnt/archive
sudo mount -o loop,ro disk/archive.img /mnt/archive
df -h /mnt/archive
cat /mnt/archive/receipts-2023/2023-12.txt
cd ~ && sudo umount /mnt/archive
```

Your answer: December 2023 total: _____. Why must you `cd` out of `/mnt/archive` before unmounting? _____

Wrap-up

17:00. Marta reads your reports folder. "Sam took three weeks to not do this."

Check your deliverables:

```
ls -l ~/cyberx/work/reports ~/cyberx/work/backup
```

- [ ] BrewBot config fixed (Mission 2)
- [ ] Top-seller list (Mission 3)
- [ ] Attacker IP found and saved in `attackers.txt` (Missions 4 and 6)

- [ ] brewlogd stopped (Mission 5)
- [ ] `vendor.log` redacted — no `@` left (Mission 6)
- [ ] Old logs archived in `work/backup` and removed (Mission 6)

Three habits that save you:

1. **man first** — `man -k` finds the command, `man` explains it.
2. **pwd after cd** — know where you are before you change anything.
3. **-i before you overwrite or delete** — `cp -i`, `mv -i`, `rm -i`. There is no recycle bin.

Homework — try it now. Each command combines things you used today. Before running each one, say out loud what you expect it to do.

```
man -k compress
mkdir -p ~/p2/a/b && cd ~/p2 && pwd
cp /etc/services . && ls -l
grep -c tcp services
tail -n 3 services
ps -ef --forest | less
df -h; du -sh ~/p2
tar -czvf p2.tgz ~/p2
rm -ri ~/p2
```

Bonus challenges (if you finish early):

- Print total revenue per day from `orders.csv`. (Hint: `awk -F, 'NR>1 {s[$6]+=$4*$5} END {for (d in s) print d, s[d]}'`)
- Find the hour of 20 October with the most declined payments, using only `grep`, `cut`, `sort` and `uniq`.
- Extract `data/mystery2` into `work/` and read `pos.ini`. Where was the old till?
- Use `find logs -size +1M` with `xargs` to list large files with their sizes.
- Apply all four config fixes from Mission 2 with one `sed -i` command on a fresh copy of the original.

To practise again from scratch, `cyberx-setup` rebuilds your lab. To remove it: `cyberx-setup --reset`.